REPUBLIC POLYTECHNIC

Week 9

C237:
Software Application Development

LESSON 18:

Database Integration with Express II – Activity

Recap

REPUBLIC POLYTECHNIC

Route to retrieve ALL products

```
// Define routes
app.get('/', (req, res) => {
    const sql = 'SELECT * FROM products';
    // Fetch data from MySQL
    connection.query( sql , (error, results) => {
        if (error) {
            console.error('Database query error:', error.message);
            return res.send('Error Retrieving products');
        }
      // Render HTML page with data
      res.render('index', { products: results });
    });
});
```

Route to retrieve ONE product by id

```
app.get('/product/:id', (req, res) => {
    // Extract the product ID from the request parameters
    const productId = req.params.id;
    const sql = 'SELECT * FROM products WHERE productId = ?';
    // Fetch data from MySQL based on the product ID
    connection.query( sql , [productId], (error, results) => {
        if (error) {
            console.error('Database query error:', error.message);
            return res.send('Error Retrieving product by ID');
        }
        // Check if any product with the given ID was found
        if (results.length > 0) {
            // Render HTML page with the product data
            res.render('product', { product: results[0] });
        } else {
            // If no product with the given ID was found
            res.send('Product not found');
        }
    });
});
```

REPUBLIC POLYTECHNIC

Route to add product

```
app.get('/addProduct', (req, res) => {
    res.render('addProduct');
});

app.post('/addProduct', (req, res) => {
    // Extract product data from the request body
    const { name, quantity, price, image } = req.body;
    const sql = 'INSERT INTO products (productName, quantity, price, image) VALUES (?, ?, ?, ?)';
    // Insert the new product into the database
    connection.query( sql , [name, quantity, price, image], (error, results) => {
        if (error) {
            // Handle any error that occurs during the database operation
            console.error("Error adding product:", error);
            res.send('Error adding product');
        } else {
            // Send a success response
            res.redirect('/');
        }
    });
});
```

REPUBLIC POLYTECHNIC

Route to update a product - Retrieve

```
app.get('/editProduct/:id', (req,res) => {
    const productId = req.params.id;
    const sql = 'SELECT * FROM products WHERE productId = ?';
    // Fetch data from MySQL based on the product ID
    connection.query( sql , [productId], (error, results) => {
        if (error) {
            console.error('Database query error:', error.message);
            return res.send('Error retrieving product by ID');
        }
        // Check if any product with the given ID was found
        if (results.length > 0) {
            // Render HTML page with the product data
            res.render('editProduct', { product: results[0] });
        } else {
            // If no product with the given ID was found, render a 404 page or handle it accordingly
            res.send('Product not found');
        }
    });
});
```

REPUBLIC POLYTECHNIC

Route to update a product - Update

```
app.post('/editProduct/:id', (req, res) => {
    const productId = req.params.id;
    // Extract product data from the request body
    const { name, quantity, price } = req.body;
    const sql = 'UPDATE products SET productName = ? , quantity = ?, price = ?  WHERE productId = ?';

    // Insert the new product into the database
    connection.query( sql , [name, quantity, price, productId], (error, results) => {
        if (error) {
            // Handle any error that occurs during the database operation
            console.error("Error updating product:", error);
            res.send('Error updating product');
        } else {
            // Send a success response
            res.redirect('/');
        }
    });
});
```

REPUBLIC POLYTECHNIC

Route to delete a product

```
app.get('/deleteProduct/:id', (req, res) => {
    const productId = req.params.id;
    const sql = 'DELETE FROM products WHERE productId = ?';
    connection.query( sql , [productId], (error, results) => {
        if (error) {
            // Handle any error that occurs during the database operation
            console.error("Error deleting product:", error);
            res.send('Error deleting product');
        } else {
            // Send a success response
            res.redirect('/');
        }
    });
});
```

Application Flow Summary

- Every **CRUD** feature follows the **application flow**:
  - User Action → Route → SQL Query → Database → Response

User Action	GET Route (Display)	POST Route (Submit)	Database Operation	Expected Result
View Products	`(GET) /`	-	SELECT	Display task list
View Product Details	`(GET) /product/:id`	-	SELECT	Display product details
Add Product	`(GET) /addProduct`	`(POST) /addProduct`	INSERT	Redirect to /
Edit Product	`(GET) /editProduct/:id`	`(POST)/editProduct/:id`	UPDATE	Redirect to /
Delete Product	`(GET) /deleteProduct/:id`	-	DELETE	Redirect to /

Displaying images

Displaying Images

- Update the app.js file to include the following line of code:

```
// Set up view engine
app.set('view engine', 'ejs');
// enable form processing
app.use(express.urlencoded({
    extended: true
}));
//  enable static files
app.use(express.static('public'));

// Define routes
app.get('/', (req, res) => {
    const sql = 'SELECT * FROM products';
```

- This line of code configures Express.js to serve static files from a directory named "**public**".

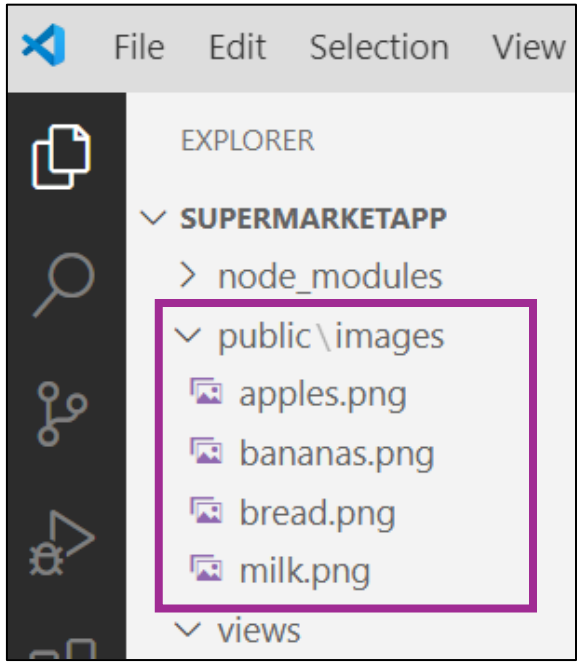
Displaying Images

- **express.static()** is a built-in middleware function in Express.js.
  - It is used to serve static files, such as **images**, CSS files, JavaScript files, and other resources, from a specified directory.
- **'public'** is the directory from which the static files will be served.
  - In this case, it's specified as "**public**", but you can replace it with any directory path relative to the root directory of your Node.js application.

```
// Set up view engine
app.set('view engine', 'ejs');
//  enable static files
app.use(express.static('public'));
```

Displaying Images

- Copy the ‘public’ folder found in your **L18 Activity Files** folder and paste it into your supermarket App folder.
- Your project folder should look like this:



Displaying Images

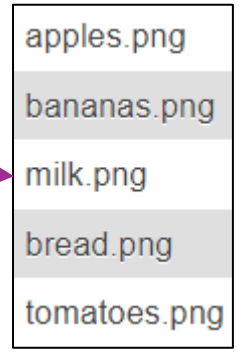
- Update your **index.ejs** page with the following code:

```
<thead>
    <tr>
      <th width="100">Product Name</th>
      <th width="100">Product Image</th>
      <th width="50">Quantity</th>
      <th width="50">Price</th>
    </tr>
</thead>
<tbody>
  <% for(let i=0; i < products.length; i++) { %>
    <tr>
      <td><%= products[i].productName %></td>
      <td><img src = "images/<%= products[i].image %>" width="20%"></td>
      <td><%= products[i].quantity %></td>
      <td>$<%= products[i].price.toFixed(2) %></td>
    </tr>
  <% } %>
</tbody>
```

Displaying Images

- Update the image name in the products table in MySQL Workbench:

productId	productName	quantity	price	image
1	Apples	100	1.50	apples.png
2	Bananas	75	0.80	bananas.png
3	Milk	50	3.50	milk.png
4	Bread	80	1.80	bread.png
5	Tomatoes	80	1.00	tomatoes.png
NULL	NULL	NULL	NULL	NULL



Let's test it out

- Save all your files.
- Open your terminal and enter "**npx nodemon app.js**" (if your server is not already running.)
- Open your browser and enter the url http://localhost:3000. You should see the following:

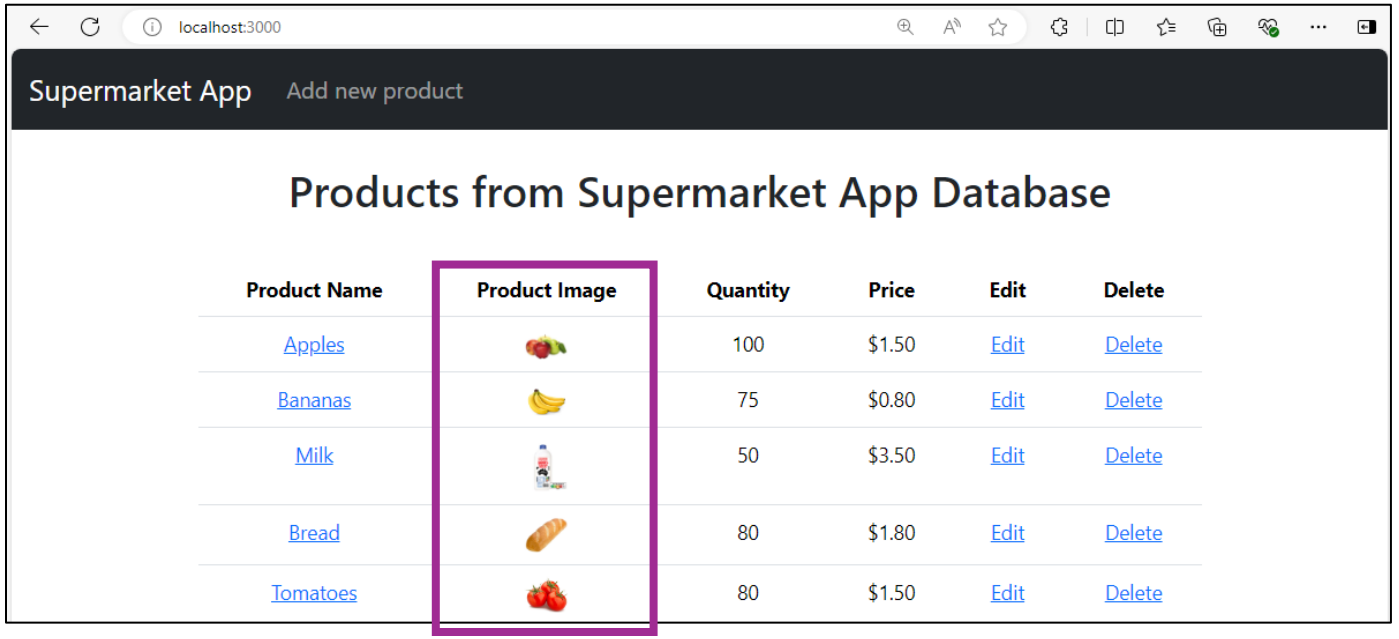


Image Upload

Steps to upload image to server

- To enable image upload to server, you can use the **multer middleware** to handle file uploads.
  1. **Install Multer package using npm:**
     npm install multer

  2. **Set Up Multer:**
     Configure multer to handle file uploads. Specify the storage location and file naming convention.

  3. **Update the route:**
     Update the route to handle both file uploads and form data.

Set up multer for file upload

- Update **app.js** to set up **multer** for file uploads:

```
const express = require('express');
const mysql = require('mysql2');
const multer = require('multer');
const app = express();

// Set up multer for file uploads
const storage = multer.diskStorage({
    destination: (req, file, cb) => {
        cb(null, 'public/images'); // Directory to save uploaded files
    },
    filename: (req, file, cb) => {
        cb(null, file.originalname);
    }
});

const upload = multer({ storage: storage });

const connection = mysql.createConnection({
    host: 'localhost'
```

Update the route (/addProduct)

- Update the route **/addProduct** in app.js to handle file upload:

```
app.post('/addProduct', upload.single('image'), (req, res) => {
    // Extract product data from the request body
    const { name, quantity, price, image } = req.body;
    const { name, quantity, price } = req.body;
    let image;
    if (req.file) {
        image = req.file.filename; // Save only the filename
    } else {
        image = null;
    }
    const sql = 'INSERT INTO products (productName, quantity, price, image) VALUES (?, ?, ?, ?)';
    // Insert the new product into the database
    connection.query( sql , [name, quantity, price, image], (error, results) => {
        if (error) {
```

The **upload.single('image')** middleware function in the multer package is used to handle single file uploads in an Express.js application.

Update views (addProduct.ejs)

- Update **addProduct.ejs** to include file upload input type:

```
<form action="/addProduct" method="POST" enctype="multipart/form-data">
    // product name form input
    // quantity form input
    // price form input
    <div class="mb-4">
      <label for="image" class="form-label">
        Image
      </label>

      <!-- <input type="text" class="form-control" id="image" name="image" placeholder="Enter image URL" required> -->
      <input type="file" class="form-control" id="image" name="image" accept="image/*">
    </div>

    <div class="d-grid gap-2">
      <button type="submit" class="btn btn-success">
        Add Product
      </button>

      <a href="/" class="btn btn-secondary">
        Cancel
      </a>
    </div>
</form>
```

Let's test it out

- Save all your files.
- Open your terminal and enter "**npx nodemon app.js**" (if your server is not already running.)
- Open your browser and enter the url http://localhost:3000 and try adding a new product. You should browse for an image from your storage for upload.

Update the route (/editProduct)

- Update the **post** route for **/editProduct** in **app.js** to handle file upload:

```
app.post('/editProduct/:id', upload.single('image'), (req, res) => {
    const productId = req.params.id;
    // Extract product data from the request body
    const { name, quantity, price } = req.body;
    let image  = req.body.currentImage; //retrieve current image filename
        if (req.file) { //if new image is uploaded
            image = req.file.filename; // set image to be new image filename
        }
    const sql = 'UPDATE products SET productName = ? , quantity = ?, price = ?, image =? WHERE productId = ?';

    // Insert the new product into the database
    connection.query( sql , [name, quantity, price, image, productId], (error, results) => {
        if (error) {
            // Handle any error that occurs during the database operation
```

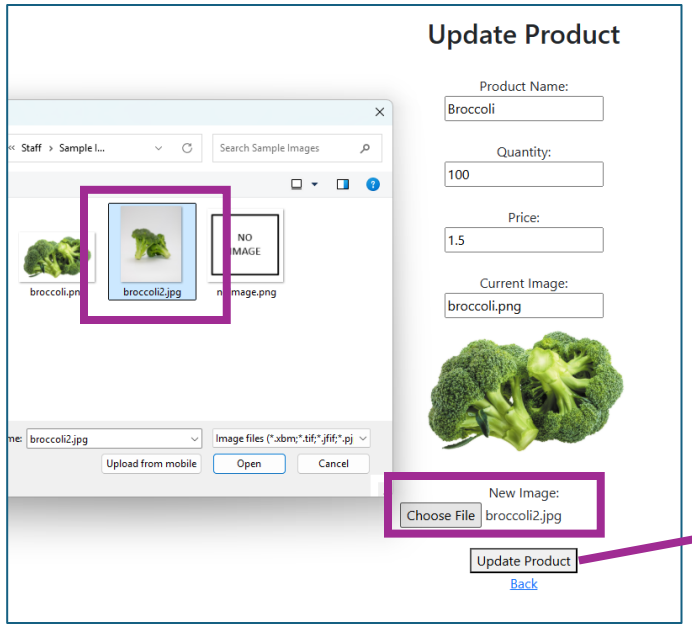
Update views (editProduct.ejs)

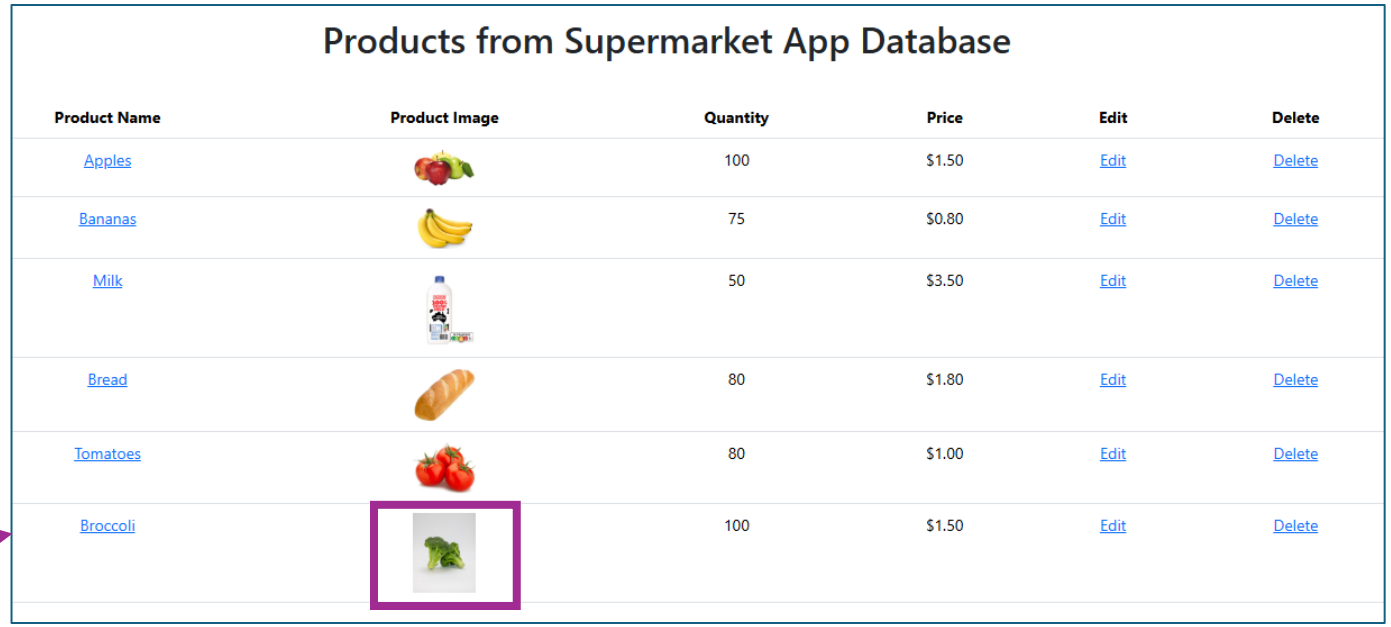
- Update **editProduct.ejs** to include file upload input type:

```
<form action="/editProduct/<%= product.productId %>" method="POST" enctype="multipart/form-data">
    // product name form input
    // quantity form input
    // price form input
    <div class="mb-3">
      <label class="form-label">
        Current Image
      </label>

      <div class="text-center">
      <input type="text" name="currentImage" value = <%= product.image %> readonly><br>
      <img src = "/images/<%= product.image %>" width="20%"><br><br>


      New Image:<br> <input type="file" id="image" name="image" accept="image/*" value=<%= product.image %> ><br><br>
      </div>
    </div>

    <div class="d-grid gap-2">
      <button type="submit" class="btn btn-warning">
        Update Product
      </button>
      <a href="/" class="btn btn-secondary">
        Cancel
      </a>
    </div>
</form>
```

Let's test it out

- Save all your files.
- Open your terminal and enter "**npx nodemon app.js**" (if your server is not already running.)
- Open your browser and enter the url http://localhost:3000 and try adding a new product. You should browse for an image from your storage for upload.





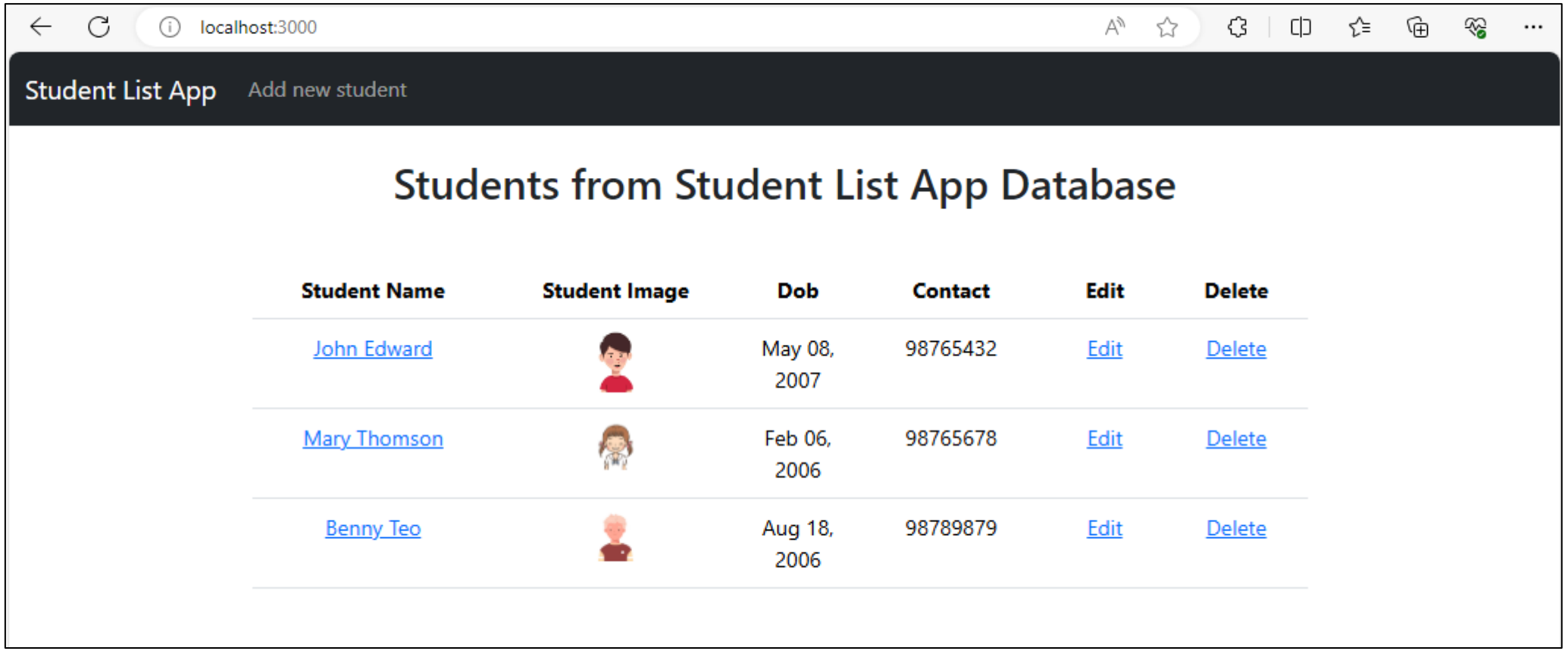
Week 09 Activity

Week 09 Activity

- Update your **StudentListApp** created in Week 08 to include the update and delete function.
- The final application should have the following functionality:
  - **Display all students'** details found in the database.
  - Display a form to **add new student** details into the database
  - **Add** new student details into the database
  - **Update** student details into database
  - **Delete** student from database
  - Upload and display a student photo with each student's details (**image upload and display**)
  - Your page should include **bootstrap navbar.**

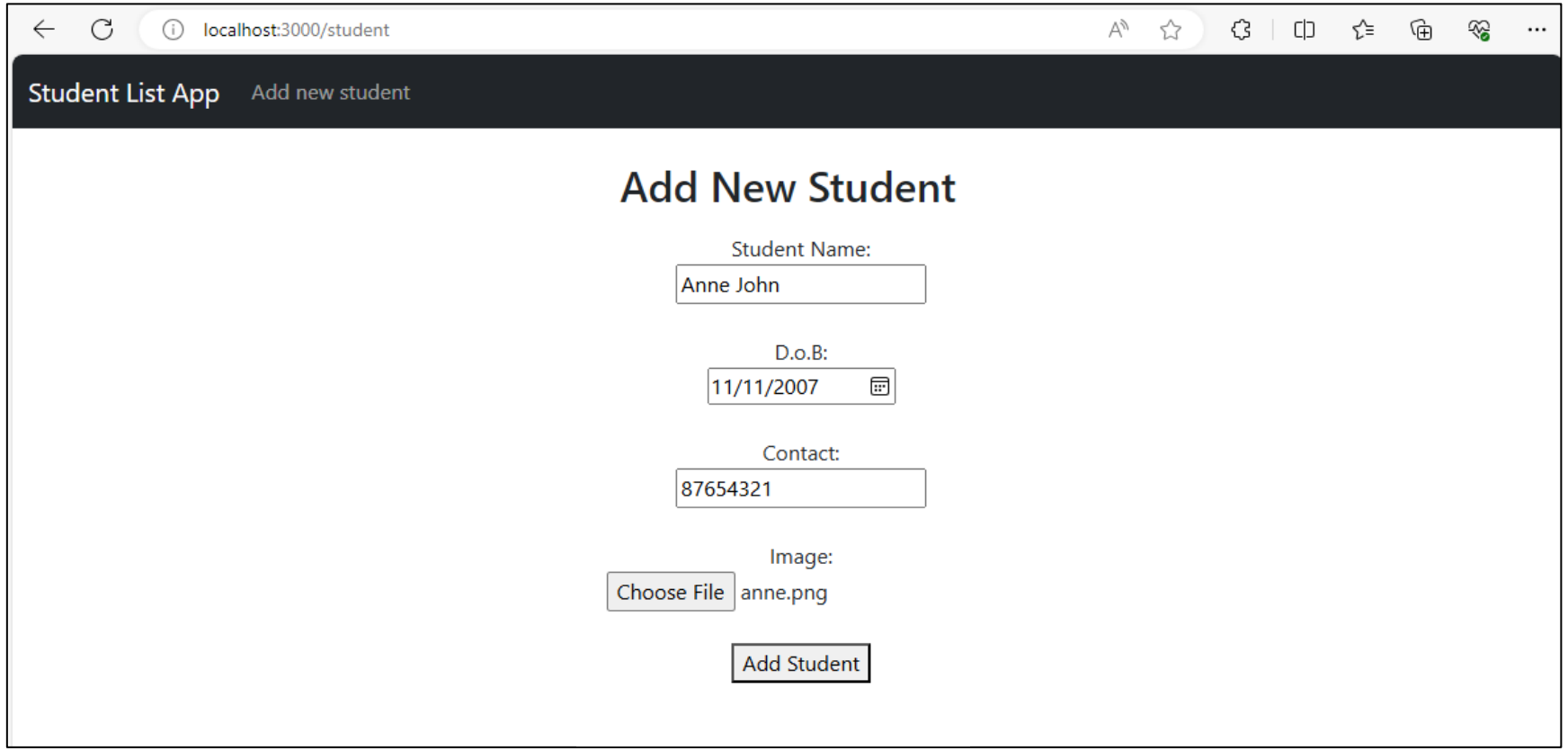
Week 09 Activity Sample Screenshots

- View all students (**index.ejs**)



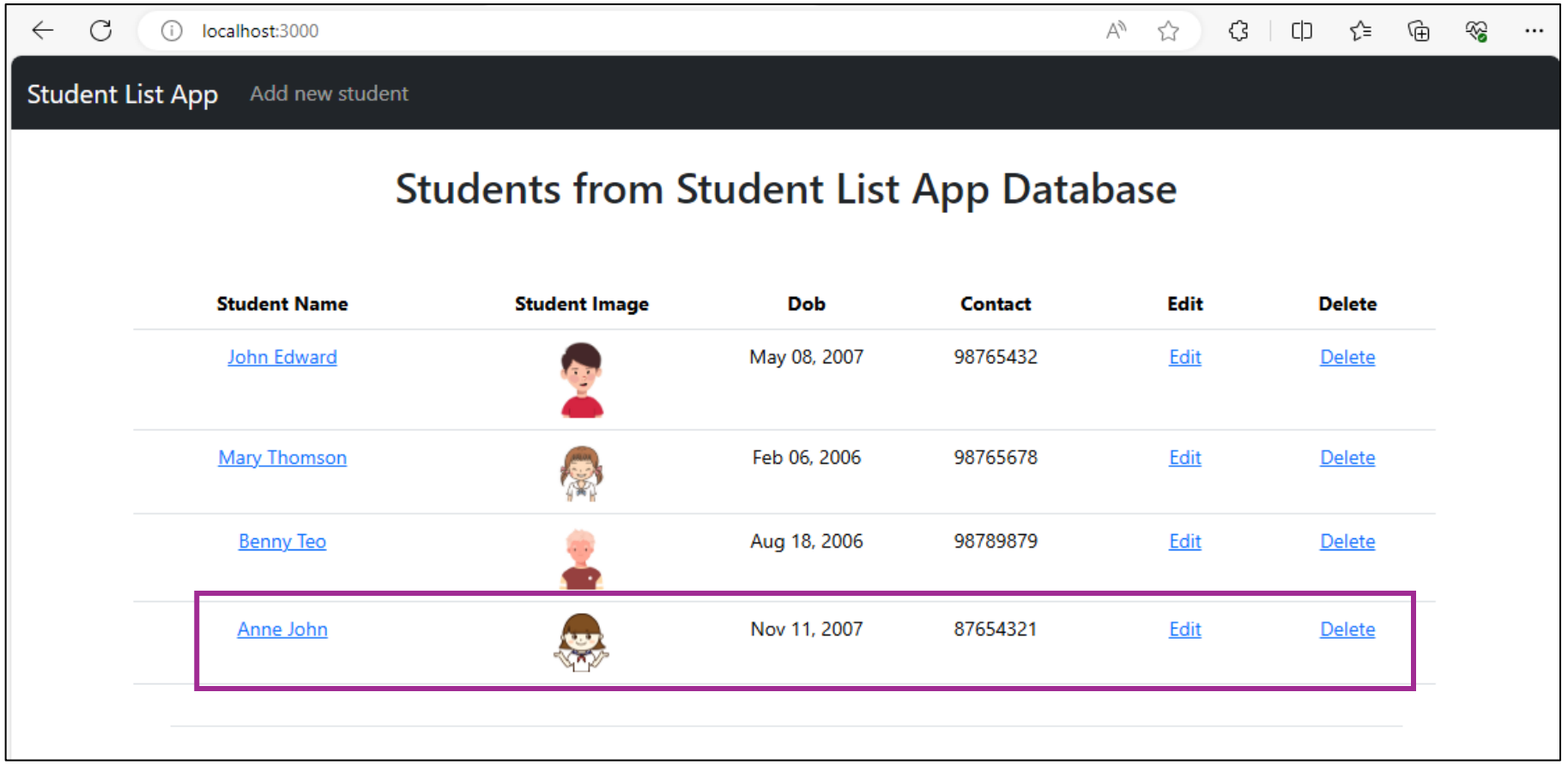
Week 09 Activity Sample Screenshots

- Add New Student (**addStudent.ejs**)



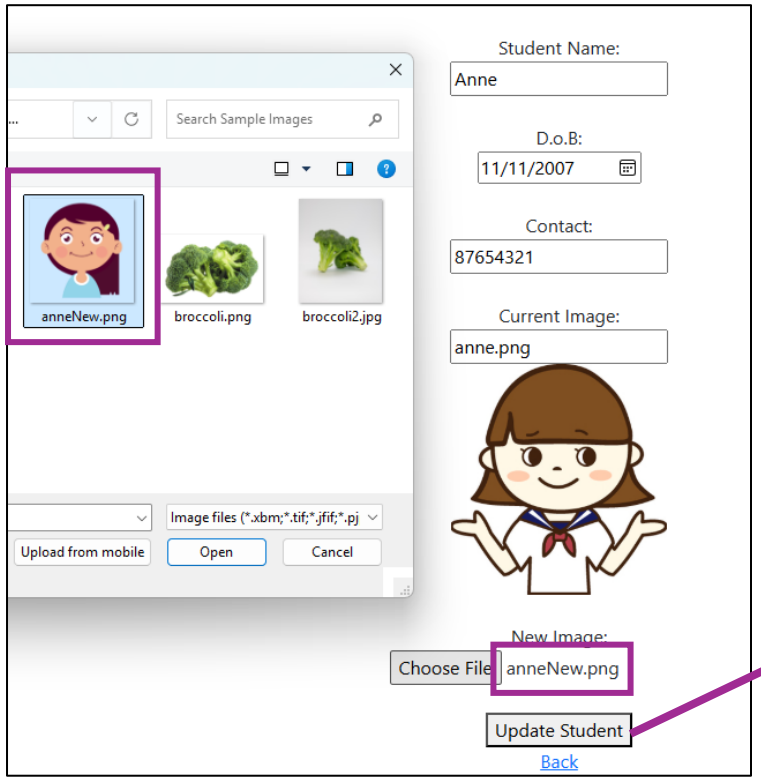
Week 09 Activity Sample Screenshots

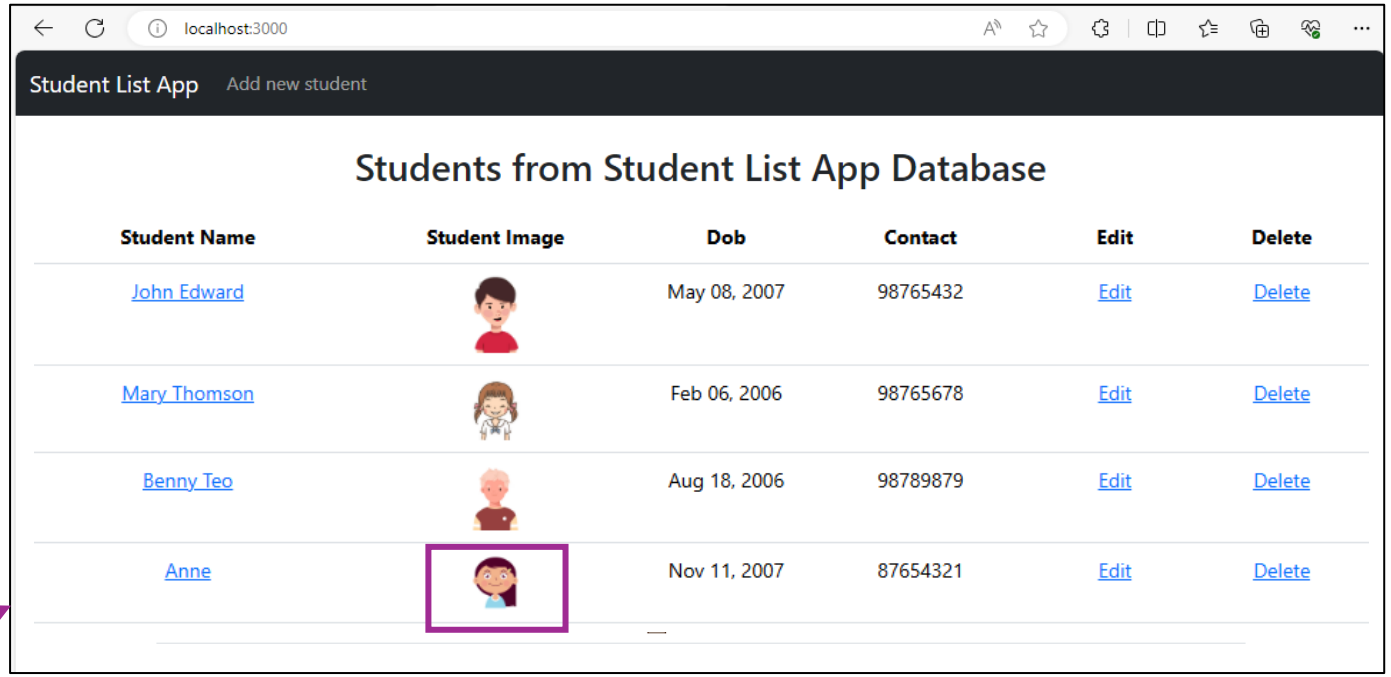
- View all students (**index.ejs**) – After new student added



Week 09 Activity Sample Screenshots

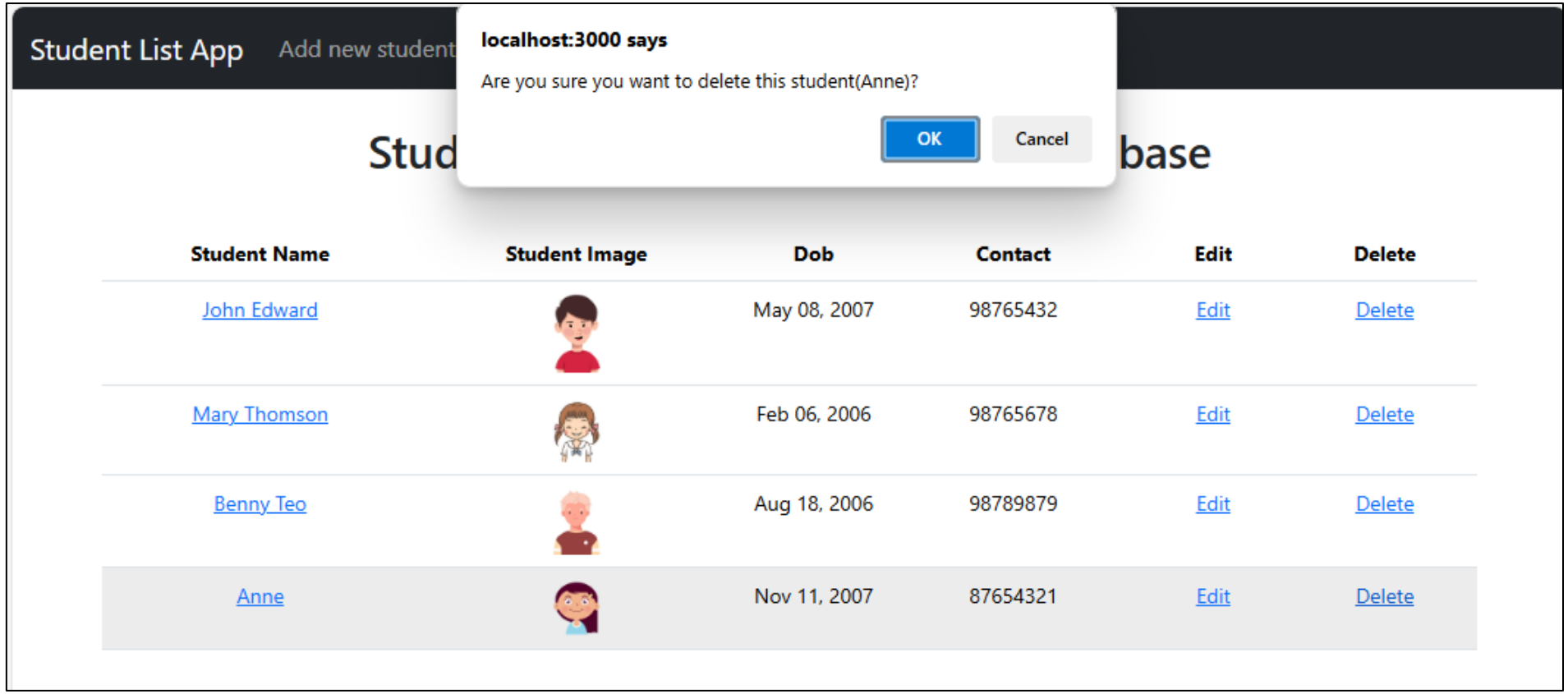
- Update student details (**updateStudent.ejs**)



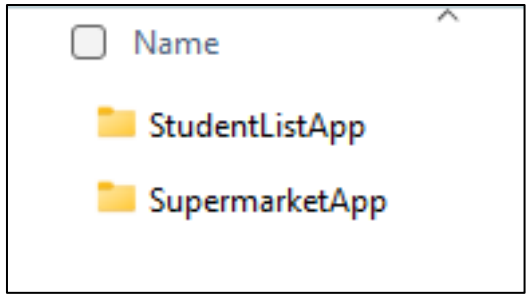


Week 09 Activity Sample Screenshots

- Delete Student (**index.ejs**) – Anne will be removed when “ok” is clicked



Deliverables

- Activity
  - 

- **Commit and push** your code to **GitHub**
- Upload all deliverables in a zipped folder to POLITEMall **before Saturday, 2359hrs.**

What have you learnt?

- Database with Express and EJS
  - Displaying Image
  - Image Upload

REPUBLIC POLYTECHNIC

End of Lesson 18